



VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

A szoftveralkalmazás dokumentációja
Vendora Webshop

Készítették:

RÉVAY ÁRON
SZŐNYE LAURA
WUNDERLICH ÁKOS

SZOMBATHELY

2026

Tartalom

Tartalom	1
Bevezetés	2
Szoftver célja	3
Komponenseinek technikai leírása	3
Működésének műszaki feltételei	5
Fejlesztői környezet beállítása (Telepítési útmutató)	5
API Dokumentáció (Végpontok)	7
Hitelesítés és Biztonság (Authentication)	8
Frontend architektúra	8
Használt technológiák és keretrendszerek	11
Backend technológiák és architektúra	11
Adatmodell	13
Egységtesztek	19
Monitor Képek	20
Főoldal	20
Fiókom menü	23
Vezérlőpult/Termékeim(Admin/Vendor jogosultság)	24
Regisztráció	28
Bejelentkezés	30
Összegzés	32
Források	32
Továbbfejlesztési lehetőségek	33

Bevezetés

A Program a Vas Vármegyei SZC Horváth Boldizsár Közgazdasági és Informatikai Technikum 13. evfolyaman szoftverfejlesztő és -tesztelők vizsgaremek céljából jött létre a 2025-2026-os tanévben.

Napjaink e-kereskedelmi piacán számtalan webáruházzal találkozhatunk, azonban a mennyiség gyakran a minőség rovására megy. Az online térben böngészve mindennapos jelenséggé váltak az időt múlt, nehezen átlátható webdesignok, valamint a vásárlói élményt sokszor inkább rontó, erőszakos és felesleges chatbotok jelenléte. Ezek a tényezők nemcsak frusztrálják a felhasználókat, hanem a vásárlási folyamatot is indokolatlanul megbonyolítják.

Csapatunkat pontosan ez a probléma inspirálta a **Vendora** webáruház megalkotásakor. Célunk egy olyan modern, letisztult és felhasználó-központú platform létrehozása volt, amely szakít a felesleges „zajjal” és a túlszűfolt felületekkel. A *vendora* esetében a fókusz a sebességen, a logikus navigáción és a kompromisszummentes vásárlói élményen (UX) van, mellőzve minden olyan zavaró elemet, amely elvonná a figyelmet a lényegről: magáról a termékről és a zökkenőmentes vásárlásról.

A projekt során nemcsak a vizuális megjelenés frissítésére fektettünk hangsúlyt, hanem egy stabil és jövőtálló technológiai alap kiépítésére is. A hagyományos, sokszor lassú megoldások helyett a kliensoldalt modern, Single Page Application (SPA) architektúrára építettük, amelyet egy biztonságos és gyors .NET alapú háttérrendszer szolgál ki.

Ez a dokumentáció bemutatja a *vendora* fejlesztésének teljes életciklusát: a kezdeti követelmények meghatározásától kezdve, a csapatmunkán és a szoftverarchitektúra tervezésén át, egészen a megvalósítás és tesztelés folyamatáig.

Szoftver célja

Egy létező ötlet újra dolgozása, célunk volt, hogy ne próbáljuk meg újra feltalálni a kereket. A *vendora* webáruház fejlesztésének elsődleges célja egy olyan modern, e-kereskedelmi platform létrehozása volt, amely mind a vásárlók, mind pedig az üzemeltetők (adminisztrátorok) számára gyors, biztonságos és kompromisszummentes használatot biztosít. A szoftver célkitűzéseit két fő felhasználói csoportra bontva határoztuk meg:

1. Vásárlói (Kliens) célok:

Zökkenőmentes vásárlási élmény (UX): A rendszer célja, hogy a vásárló a lehető legkevesebb kattintással és várakozási idővel találja meg a keresett terméket, és tudja azt a kosarába helyezni. A felesleges vizuális elemek, pop-upok és chatbotok elhagyásával a fókusz teljes mértékben a termékkínálaton marad.

Gyorsaság és reszponzivitás: A Single Page Application (SPA) kialakítás révén a szoftver célja a villámgyors oldalváltás biztosítása, valamint a tökéletes megjelenés asztali számítógépen, tableten és mobil eszközön egyaránt.

Adatbiztonság: A vásárlók személyes adatainak (pl. jelszavak, rendelési információk) maximális védelme iparági standardok (pl. jelszó-hashelés) alkalmazásával.

2. Üzemeltetői (Adminisztrátori) célok:

Egyszerű tartalomkezelés: Az adminisztrátorok számára a szoftver egy könnyen átlátható felületet biztosít, ahol technikai tudás nélkül is lehetséges az új termékek feltöltése, a meglévők módosítása vagy törlése (CRUD műveletek).

Rendelések nyomon követése: A beérkező vásárlások státuszának áttekintése és menedzselése egy központi felületről.

Skálázhatóság: A szoftver architektúrája úgy lett megtervezve, hogy a jövőben a termékkínálat vagy a felhasználószám növekedése esetén is stabil maradjon a kiszolgálás.

Komponenseinek technikai leírása

A *vendora* webáruház egy klasszikus **kliens-szerver (Client-Server)** architektúrára épül, ahol a megjelenítésért felelős frontend, és az üzleti logikát, valamint az adat elérést biztosító backend élesen elkülönül egymástól. Ez a szeparáció biztosítja a kód tisztaságát, karbantarthatóságát és a rendszer biztonságát.

A szoftver logikai felépítése az alábbi főbb komponensekből áll:

1. Kliensoldali (Frontend) komponensek: Ez a réteg felel a felhasználóval való interakcióért. A React alapú felület kisebb, újrahasonosítható építőkockákból (komponensekből) áll össze.

Állapotkezelés (State Management): Az alkalmazás folyamatosan nyomon követi a felhasználó aktuális állapotát (pl. be van-e jelentkezve, mik vannak a kosarában). Ezek az adatok a memóriában tárolódnak a böngészés során.

Útválasztó (Router) komponens: A *react-router-dom* felel azért, hogy az URL változásakor a megfelelő nézet (pl. */termek*, */kosar*, */admin*) töltsön be az oldal újratöltése nélkül.

Nézetek és UI komponensek: Az egyedi felületelemek, mint a navigációs sáv (Navbar), a termékkártyák (ProductCard), vagy a bejelentkezési űrlapok.

2. Szerveroldali (Backend) komponensek: A .NET Web API réteg a Model-View-Controller (MVC) mintára épülve fogadja a kliensoldalról érkező HTTP kéréseket.

Végpontok (Controllers): Ezek a komponensek fogadják a frontend felől érkező kéréseket (pl. GET kérés az összes termék lekérésére, vagy POST kérés új felhasználó regisztrálására).

Adatátviteli objektumok (DTO - Data Transfer Objects): A biztonság érdekében a rendszer sosem küldi el a teljes adatbázis-modellt a kliensnek. A DTO-k felelnek azért, hogy csak a szükséges és publikus információk (pl. termék neve és ára, de az adatbázis belső ID-ja vagy a létrehozás dátuma nem feltétlenül) utazzanak a hálózaton.

Adatelérési réteg (Entity Framework DbContext): Ez a komponens fordítja le a C# kódban megírt parancsokat SQL lekérdezésekké, és hajtja végre azokat a MySQL adatbázison.

Működésének műszaki feltételei

Hardver Követelmények (Minimum):

- **Processzor (CPU):** 1 vCPU (lokális teszteléshez elegendő bármilyen modern, 10 évnél nem régebbi processzor).
- **Memória (RAM):** 1-2 GB (az operációs rendszer és a webservert együttes futtatásához).
- **Tárhely:** Legalább 1 GB szabad tárhely (a forráskód, az adatbázis és a feltöltött termékképek tárolására).

Szoftverkörnyezet (Lokális futtatás esetén):

- **Webszerver csomag:** XAMPP (vagy azzal egyenértékű megoldás, pl. WAMP, MAMP).
- **Webszerver:** Apache.
- **Programnyelv:** PHP (ajánlott verzió: 8.x vagy újabb).
- **Adatbázis-kezelő:** MySQL (8.x) vagy MariaDB.

Támogatott böngészők (aktuális főverziók):

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Apple Safari

Szoftveres feltételek a böngészőben:

- A JavaScript futtatásának engedélyezése (a dinamikus UI elemek és a kosár működéséhez).
- Reszponzív felbontás támogatása: Az oldal mobil, tablet és asztali (minimum 320px széles) képernyőkön is használható.

Fejlesztői környezet beállítása (Telepítési útmutató)

Ez a fejezet leírja a szoftver helyi (lokális) futtatásához szükséges lépéseket és technikai feltételeket.

A fejlesztés megkezdése előtt győződjön meg róla, hogy a következő szoftverek telepítve vannak a számítógépén:

1. Verziókezelés:

- Git (a forráskód letöltéséhez és kezeléséhez)

2. Futtatókörnyezetek (Runtimes & SDK-k):

- Node.js: (Ez tartalmazza az NPM csomagkezelőt is, ami a React és a Vite futtatásához szükséges).
- .NET SDK: (A C# és ASP.NET Core backend fordításához és futtatásához).

3. Fejlesztői környezetek (IDE / Kódszerkesztők):

- **Visual Studio 2022: A backend (ASP.NET) fejlesztéséhez ajánlott környezet.**
- **Visual Studio Code:** A frontend (React) fejlesztéséhez ajánlott szerkesztő.

4. Szükséges Csomagok a futtatáshoz:

- BCrypt.Net-Next 4.1.0
- coverlet.collector 6.0.0 10.0.0
- Microsoft.AspNetCore.Authentication.JwtBearer 8.0.10 10.0.7
- Microsoft.EntityFrameworkCore.InMemory 8.0.18 10.0.7
- Microsoft.EntityFrameworkCore.SqlServer 8.0.10 10.0.7
- Microsoft.IdentityModel.Tokens 8.0.10 8.17.0
- Pomelo.EntityFrameworkCore.MySql 8.0.2 9.0.0
- Swashbuckle.AspNetCore 6.6.2 10.1.7
- System.IdentityModel.Tokens.Jwt 8.1.0 8.17.0
- Microsoft.NET.Test.Sdk 17.8.0 18.4.0
- Moq 4.20.72
- xunit 2.5.3
- xunit.runner.visualstudio 2.5.3 3.1.5

5. Adatbázis és eszközök:

- **MySQL 8.0: Az adatbázis motor.**
- **phpMyAdmin:** (Vagy MySQL Workbench) Az adatbázis vizuális kezeléséhez.

6. A rendszer lokális indítása

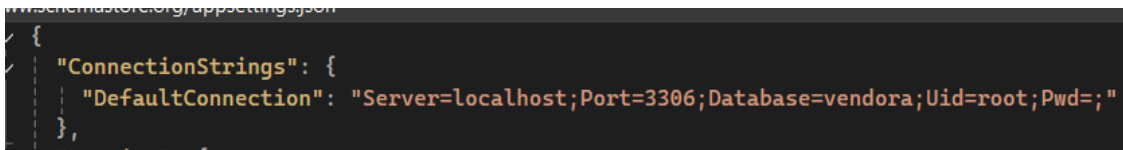
7. Az adatbázis beállítása (XAMPP)

- Indítsa el a XAMPP vezérlőpultot, és indítsa el az Apache és MySQL modulokat.

- Nyissa meg a böngészőben a phpMyAdmin felületét (alapértelmezetten: <http://localhost/phpmyadmin>).
- Hozzon létre egy új adatbázist a projekt számára (CREATE DATABASE vendora;).
- Az "Importálás" (Import) menüponton keresztül válassza ki és importálja a letöltött repositoryban található `.sql` fájlt, amely létrehozza a szükséges táblákat és betölti az alapértelmezett adatokat.

8. A Backend indítása (Visual Studio 2022)

- Navigáljon a letöltött repository backend könyvtárába.
- Nyissa meg a projektet a `VendoraAPI.sln` fájlra kattintva, amely elindítja a Visual Studio 2022-t.
- Futtassa a projektet (F5 vagy a Start gomb használatával).
- A sikeres indulást követően a böngészőben automatikusan megnyílik a Swagger UI dokumentációs és tesztelő felülete.
- Az API végpontok használatához kattintson az *Authorize* gombra, és végezze el a hitelesítést az API kulcs megadásával.
- Fontos, hogy a Backendben, az `""appsettings.Development.json""`-ben a portszám egyezzen a XAMPP-ban indított MySQL port számával egyezik, mi esetünkben ez **3306**



```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;Port=3306;Database=vendora;Uid=root;Pwd=;"
  }
}

```

9. A Frontend indítása (Visual Studio Code)

- Nyissa meg a repository `frontend` mappáját a Visual Studio Code-ban.
- Nyissa le az integrált terminált (Terminal > New Terminal).
- A fejlesztői szerver indításához futtassa az alábbi parancsot: `npm run dev`
- Sikeres indítás után a terminálban nyomja meg az `o` billentyűt (majd Entert), amely automatikusan megnyitja a csatlakoztatott weboldalt az alapértelmezett böngészőben.

API Dokumentáció (Végpontok)

Interaktív dokumentáció (Swagger UI)

A rendszer API végpontjainak naprakész, részletes listája automatikusan generálódik a Swagger segítségével.

- **Elérhetőség:** A backend elindítása után a dokumentáció a böngészőben érhető el (pl. <https://localhost:7211/swagger>).
- A felületen nemcsak a végpontok címei, elvárt paraméterei és válasz-modelljei tekinthetők meg, hanem a Try it out gomb segítségével azonnal tesztelhetők is.

Alap URL és adatformátum

- **Adatformátum:** A kliens és a szerver közötti kommunikáció szabványos **JSON** formátumban történik. Minden kérés törzsének (Body) és válaszának ez a formátuma.
- **Alap URL fejlesztői környezetben:** Minden API végpont a bázis URL-hez képest relatíve értendő (pl. <https://localhost:7211/api/>).

Hitelesítés és Biztonság (Authentication)

A rendszer védett végpontjainak (pl. profil módosítása, saját termékek kezelése) eléréséhez érvényes API kulcsra (vagy JWT tokenre) van szükség.

- A hitelesítési kulcsot a HTTP kérések fejlécében (Header) kell elküldeni.
- A Swagger UI felületen a hitelesítés az oldal tetején található **Authorize** gombra kattintva, a kulcs megadásával aktiválható. Ezt követően a Swagger minden további tesztkéréshez automatikusan hozzáfűzi a szükséges fejléceket.

Használt HTTP metódusok (CRUD műveletek)

Az API a szabványos REST konvenciókat követi az adatok kezelésére:

- **GET:** Adatok lekérése (pl. termékek listázása, kategóriák lekérése). Ez a metódus nem módosít adatot a szerveren.
- **POST:** Új erőforrás létrehozása (pl. új felhasználó regisztrációja, új termék feltöltése).
- **PUT / PATCH:** Meglévő adatok teljes vagy részleges módosítása, frissítése.
- **DELETE:** Erőforrás törlése a rendszerből.

Frontend architektúra

A kliens oldali alkalmazás egy modern React alapú egyoldalas webalkalmazás (SPA), amely a felhasználói felület megjelenítéséért és a backenddel való kommunikációért felel.

Használt főbb technológiák

- **Keretrendszer:** React
- **Útvonalválasztás (Routing):** `react-router-dom` a kliensoldali navigáció dinamikus, oldal frissítés nélküli kezelésére.
- **Állapotkezelés:** React Context API (a beépített hookok segítségével).

Mappastruktúra és architektúra

A forráskód logikai egységekre van bontva a tiszta kód és a karbantarthatóság érdekében. A `src` (forrás) mappán belüli főbb könyvtárak az alábbiak:

- **`/assets`:** A statikus fájlok (pl. képek, ikonok, globális stíluslapok) tárolási helye.
- **`/services`:** A backenddel (API) történő kommunikációért felelős réteg. Itt található a `fetchApi.js` fájl, amely központilag kezeli a HTTP kéréseket.
- **`/Components`:** Az alkalmazás összes felhasználói felületi eleme és teljes oldala (nézete) itt található, funkciók szerint külön mappákba rendezve.
 - *Példák az oldalszintű komponensekre:* `FrontPage` (Főoldal), `VendorDashboard` (Eladói vezérlőpult), `Checkout` (Fizetés), `ProductDetail` (Termékoldal).
 - *Példák a kisebb építőelemekre:* `Navbar` (Navigáció), `Footer` (Lábléc), `ThemeToggle` (Témaváltó).

Állapotkezelés (State Management)

Az alkalmazás a globális állapotok megosztására a React beépített **Context API**-jét használja, így elkerülhető a felesleges "prop drilling" (az adatok túlzott mélységű átadása a komponensek között). A legfelső szinten (az `App.js`-ben) két fő szolgáltató (Provider) öleli körbe az alkalmazást:

- **`CartProvider`:** A bevásárlókosár tartalmának (hozzáadás, törlés, mennyiség) globális memóriában tartásáért felel.
- **`WishlistProvider`:** A felhasználó kívánságlistájának (kedvelt termékek) állapotát kezeli az egész rendszerben.

Navigáció és Útvonalak (Routing)

A nézetek közötti váltást a `BrowserRouter` és a `Routes` komponensek irányítják. Az alkalmazásban definiált főbb útvonalak:

- `/` – Főoldal (`FrontPage`)

- `/dashboard` – Vezérlőpult (**VendorDashboard**)
- `/product/:id` – Dinamikus termékoldal, amely az URL-ben szereplő `id` alapján tölti be a megfelelő termék részleteit (**ProductDetail**).
- `/browse` – Böngészés (**Browse**)
- `/support` – Ügyfélszolgálat/Támogatás (**Support**)
- `/vendors` – Eladók listája (**Vendors**)
- `/account` – Fiók/Profil oldal (**Account**)
- `/checkout` – Fizetési folyamat (**Checkout**)

Globális UI Komponensek

Bizonyos komponensek az útvonalválasztón (**Routes**) kívül helyezkednek el, így azok az oldalváltásoktól függetlenül, a rendszer minden pontján folyamatosan elérhetők:

- **Cart:** A globális bevásárlókosár felülete.
- **ThemeToggle:** Az oldal megjelenését (Világos / Sötét mód) váltó gomb, amely globálisan kihat a felület stílusára.

API Kommunikáció

A frontend a backend végpontjait a `services/fetchApi.js` modulon keresztül hívja meg. Ebbe a fájlba vannak kiszervezve a szerverrel való kommunikációs logikák (HTTP GET, POST műveletek), biztosítva, hogy a vizuális komponensek kódja letisztult maradjon.

Használt technológiák és keretrendszerek

A *Vendora* webáruház modern, kliensoldal-központú architektúrára épül. A hagyományos, többoldalas weblapok (MPA - Multi-Page Application) helyett a projekt egy úgynevezett **Single Page Application (SPA)** megközelítést alkalmaz. Ennek lényege, hogy a böngésző csak egyszer tölti be az alapvető HTML, CSS és JavaScript fájlokat, a további navigáció és adatlekérés pedig a háttérben, az oldal újratöltése nélkül történik.

A kliensoldal (frontend) fejlesztése során az alábbi modern eszközöket és könyvtárakat alkalmaztuk:

- **React.js:** A felhasználói felület (UI) felépítéséért felelős, komponensalapú JavaScript könyvtár. Használata lehetővé tette a webshop felületének (pl. termékkártyák, kosár, navigációs sáv) újrahasznosítható építőkövekre bontását. Ez nemcsak a kód átláthatóságát növelte, hanem a reaktív állapotkezelés (state management) révén azonnali vizuális visszajelzést biztosít a felhasználónak (például amikor egy terméket a kosárba tesz).
- **React Router DOM:** A kliensoldali útválasztásért (routing) felelős csomag. Ennek segítségével valósult meg a navigáció az egyes aloldalak (Főoldal, Termékek, Kosár, Admin felület) között. Mivel a váltás az oldal teljes újratöltése nélkül történik, az alkalmazás használata rendkívül gyors és folyamatos, ami elengedhetetlen egy modern e-kereskedelmi platformnál.
- **Vite:** Következő generációs frontend fejlesztői és build (fordító) eszköz. A hagyományos megoldásokkal (pl. Webpack vagy Create React App) szemben a Vite azonnali szervertelítést és villámgyors Hot Module Replacement (HMR) funkciót biztosít a fejlesztés során. Élesítéskor (build) pedig rendkívül optimalizált, kisméretű és gyorsan betölthető statikus fájlokat generál az alkalmazásból.

Backend technológiák és architektúra

A *vendora* webáruház szervertoldali (backend) logikáját egy **ASP.NET Core Web API** biztosítja. Ez a réteg felel az adatok kiszolgálásáért, az üzleti logika futtatásáért és az adatbázissal való biztonságos kommunikációért. A fejlesztés során a következő kiemelt csomagokat (NuGet packages) integráltuk a rendszerbe:

- **Adatbázis-kezelés és ORM (Object-Relational Mapping):**
 - **Entity Framework Core (Microsoft.EntityFrameworkCore.Tools):** A projekt úgynevezett "Code-First" (kód-első) megközelítést alkalmaz. Ez azt jelenti, hogy az adatbázis tábláit nem manuálisan hoztuk létre, hanem a C# osztályok (modellek) alapján az Entity Framework generálta le őket. A *Tools* csomag biztosította az adatbázis-migrációk (verziókövetés) futtatását.
 - **Pomelo.EntityFrameworkCore.MySql:** Mivel a lokális környezetben a XAMPP által biztosított MySQL adatbázist használtuk, ez a nagy teljesítményű illesztőprogram felelt a .NET alkalmazás és a MySQL szerver közötti zökkenőmentes kommunikációért. *(Megjegyzés: A projektben a*

Microsoft.EntityFrameworkCore.SqlServer csomag is rendelkezésre állt alternatív adatbázis-kapcsolat céljából.)

- **Biztonság és Felhasználó-azonosítás (Authentication & Authorization):**
 - **Microsoft.AspNetCore.Authentication.JwtBearer** és **Microsoft.IdentityModel.Tokens:** Mivel a kliensoldal egy React alapú SPA (Single Page Application), a hagyományos, szerveroldali munkamenet (session) helyett a modern **JWT (JSON Web Token)** technológiát alkalmaztuk. Sikeres bejelentkezéskor az API egy titkosított tokent állít ki a kliensnek, amellyel a felhasználó a későbbiekben igazolni tudja a jogosultságait (pl. rendelés leadásakor vagy adminisztrációs feladatoknál).
 - **BCrypt.Net-Next:** A felhasználók védelme kritikus szempont egy webshopnál. A rendszer a jelszavakat sosem tárolja egyszerű szöveggént (plaintext). Helyette a BCrypt algoritmus segítségével úgynevezett "sózott" hashelést (egyirányú titkosítást) alkalmazunk, így egy esetleges adatszivárgás esetén is visszafejthetetlenek maradnak a jelszavak.
- **Fejlesztői eszközök és API Dokumentáció:**
 - **Swashbuckle.AspNetCore (Swagger):** Ez a modul egy automatikusan generált, interaktív OpenAPI dokumentációt biztosít. Fejlesztés közben felbecsülhetetlen értékű volt, mivel egy grafikus webes felületen lehetett tesztelni és áttekinteni a webshop összes elérhető API végpontját, anélkül, hogy a frontend már teljesen készen lett volna.

Konfiguráció és környezeti változók

A rendszer rugalmasságát és hordozhatóságát az **appsettings.json** fájl teszi lehetővé. Ebben a konfigurációs állományban tároljuk az adatbázis kapcsolati karakterláncát (*Connection String*), valamint a **JWT** tokenek generálásához és hitelesítéséhez szükséges kulcsot. Ez a megközelítés lehetővé teszi, hogy csak a konfiguráció átírásával, a szoftver forráskódjának módosítása nélkül, válthassunk a lokális fejlesztői környezet és a felhőalapú adatbázis-kiszolgálók között.

appsettings.json:

```
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Server=localhost;Port=3306;Database=vendora;Uid=root;Pwd="
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "AllowedHosts": "*",
12   "Jwt": {
13     "Key": "EzEgyNagyonHosszuEsTitkosSzuperKulcsAmitSenkiNemTudhat12345!",
14     "Issuer": "VendoraAPI",
15     "Audience": "VendoraAppUsers"
16   }
17 }
```

A fenti Connection String alapján csatlakozik az adatbázishoz, ha nem tud akkor hibaüzenetet ad. A Connection String megváltoztatásával könnyedén módosítható a csatlakozás, ha megváltozik például a jelszó, vagy a szerver.

- **A rendszer főbb strukturális egységei:**

- **Controllers:** Ez a réteg képezi az alkalmazás belépési pontjait. A vezérlők felelnek a HTTP kérések fogadásáért, a bejövő adatok validálásáért, valamint a megfelelő válaszok (JSON) visszaküldéséért. Például az **OrdersController** kezeli a teljes rendelési folyamatot, míg a **UsersController** a hitelesítésért felel.
- **Models:** Az adatbázis-sémát leképező C# osztályok gyűjteménye. Itt definiáltuk a rendszer alapvető entitásait, mint a **Product**, **Order**, **User** és a **Category**, valamint a köztük lévő kapcsolatokat (pl. egy rendeléshez több rendelési tétel tartozik).
- **DTOs (Data Transfer Objects):** Az adatátviteli objektumok használatával biztosítjuk, hogy a kliensoldal és a szerver között csak a szükséges adatok mozogjanak. Ez fokozza a biztonságot és csökkenti a hálózati forgalmat.
- **Data:** Az **ApplicationDbContext** osztályon keresztül valósul meg az Entity Framework Core integráció. Ez a komponens hidat képez a C# kód és a MySQL adatbázis között, transzparens módon kezelve a lekérdezéseket és az adatok perzisztenciáját.

- **Üzleti logika és folyamatok:**

A backend aktívan kezeli a webáruház működését és üzleti szabályokat érvényesít a tranzakciók során.

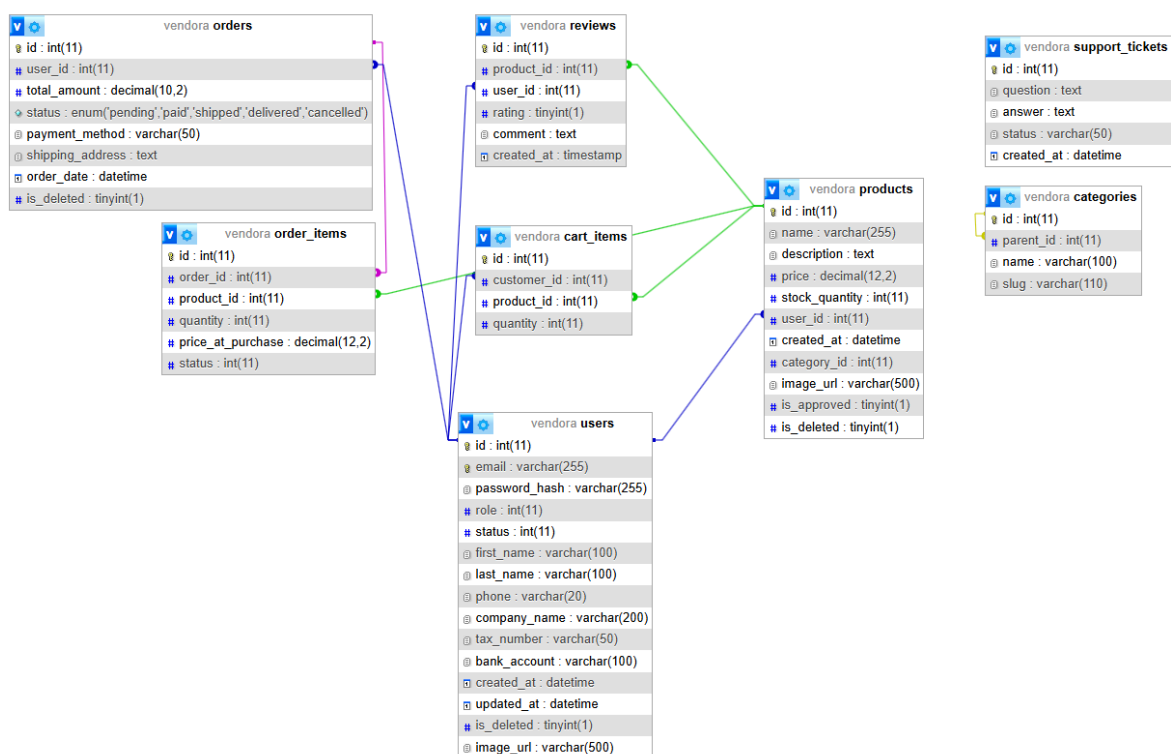
- **Rendeléskezelés:**
 - A rendelési folyamat során a rendszer automatikusan kiszámolja a végösszeget a kosár tartalma alapján, és rögzíti a vásárlás pillanatában érvényes árakat, ezzel megvédve a tranzakciót a későbbi árváltozásoktól.
- **Szerepkör alapú hozzáférés:** A biztonság érdekében a rendszer **JWT** alapú azonosítást használ, amely a szerepköröket különíti el
 - **Vásárló:** Csak a saját profiljához és rendeléseihez fér hozzá.
 - **Eladó:** Feltöltheti és kezelheti saját termékeit
 - **Adminisztrátor:** Teljes körű jogosultsággal rendelkezik a platform felett, jóváhagyhatja a jelentkező eladókat, moderálhatja a feltöltött termékeket és a **Support** fülön lévő kérdéseket válaszolhatja meg.
- **Adatvédelem:**
 - Az adatok védelme érdekében a rendszer **Soft Delete** technológiát alkalmaz. Ez azt jelenti, hogy a rendelések vagy termékek törlésekor az adatok nem lesznek törölve véglegesen az adatbázisból, hanem egy

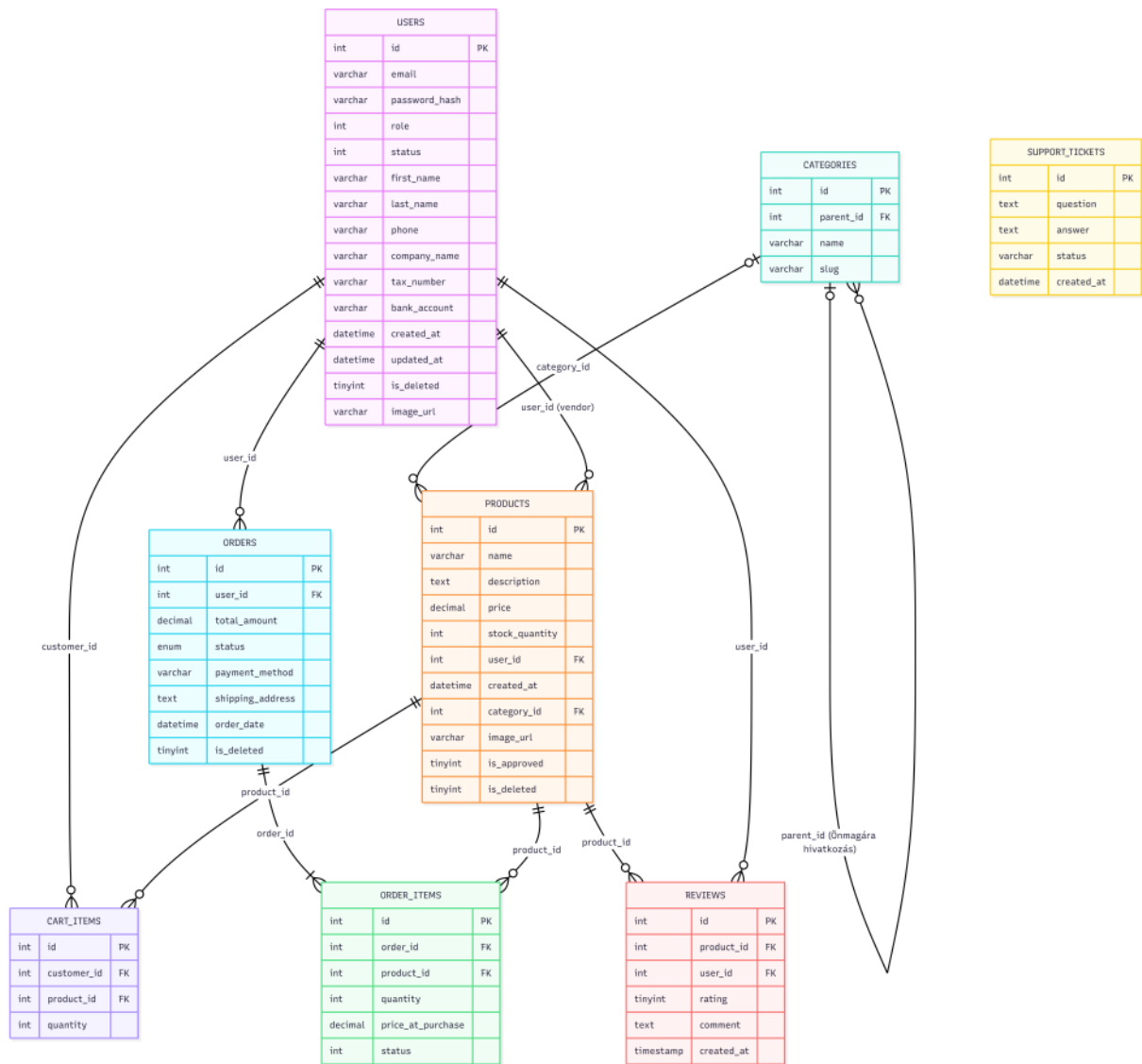
IsDeleted jelzővel ellátva láthatatlanná válnak a felhasználók számára, ezzel megőrizve a könyvelési folytonosságot.

Adatmodell

A webáruházunk adatai egy **MySQL** alapú relációs adatbázisban tárolódnak. Az adatbázis sémáját az Entity Framework Core Code-First megközelítése alapján hoztuk létre. Ez a technológia biztosítja hogy az adatszerkezet és a forráskód folyamatos szinkronban legyen.

EER Diagram:





Az adatok tárolása 8 különálló táblában történik a *vendora* adatbázisban

1. **users** tábla: A rendszer felhasználóit tartalmazza.

- *id*: Elsődleges kulcs, a felhasználó egyedi azonosítója
 - Adattípus: INT (szám)
- *email*: A felhasználó e-mail címe, egyben a bejelentkezési azonosító
 - Adattípus: VARCHAR (szöveg)
- *password_hash*: A felhasználó titkosított jelszava
 - Adattípus: VARCHAR (szöveg)
- *role*: A felhasználó szerepköre (0: Admin, 1: Vendor, 2: Customer)
 - Adattípus: INT (szám)

- *first_name / last_name* : A felhasználó teljes neve
 - Adattípus: VARCHAR (szöveg)
- *phone*: A felhasználó telefonszáma
 - Adattípus: VARCHAR (szöveg)
- *company_name*: A felhasználó cégének a neve (Csak abban az esetben ha a felhasználó eladóként regisztrált)
 - Adattípus: VARCHAR (szöveg)
- *tax_number*: A felhasználó cégének az adószáma (Csak abban az esetben ha a felhasználó eladóként regisztrált)
 - Adattípus: VARCHAR (szöveg)
- *bank_account*: A felhasználó bank számlaszámának adatai
 - Adattípus: VARCHAR (szöveg)
- *created_at*: Regisztráció dátuma
 - Adattípus: DATETIME (dátum)
- *is_deleted*: Soft-delete jelző, a felhasználó fiókja “törölve van-e”
 - Adattípus: TINYINT (igaz/hamis)

2. **products tábla:** A webáruházban elérhető termékek adatait tárolja.

- *id*: Elsődleges kulcs, a termék egyedi azonosítója
 - Adattípus: INT (szám)
- *name*: A termék megnevezése
 - Adattípus: VARCHAR (szöveg)
- *description*: A termék leírása
 - Adattípus: TEXT (szöveg)
- *price*: A termék ára
 - Adattípus: DECIMAL (tizedesponos szám)
- *stock_quantity*: Az elérhető készlet mennyisége
 - Adattípus: INT (szám)
- *user_id*: Idegen kulcs, amely a terméket az eladóhoz köti
 - Adattípus: INT (szám)
- *created_at*: A termék feltöltésének dátuma
 - Adattípus: DATETIME (dátum)

- *category_id*: Idegen kulcs, melyik kategóriába tartozik a termék
 - Adattípus: INT (szám)
- *image_url*: A termék képének elérési útja
 - Adattípus: VARCHAR (szöveg)
- *is_approved* / *is_deleted*: Jóváhagyási és törlési státuszok
 - Adattípus: TINYINT (igaz/hamis)
- 3. **categories tábla**: A termékek rendszerezésére szolgáló tábla
 - *id*: Elsődleges kulcs, a kategória egyedi azonosítója
 - Adattípus: INT (szám)
 - *parent_id*: Önhivatkozó idegen kulcs alkategóriák létrehozásához
 - Adattípus: INT (szám)
 - *name*: A kategória megnevezése
 - Adattípus: VARCHAR (szöveg)
 - *slug*: Keresőbarát útvonal au URL-ekhez
 - Adattípus: VARCHAR (szöveg)
- 4. **orders tábla**: A leadott rendeléseket adatait tárolja
 - *id*: Elsődleges kulcs, a rendelés egyedi azonosítója
 - Adattípus: INT (szám)
 - *user_id*: Idegen kulcs a rendelést leadó vásárlóhoz
 - Adattípus: INT (szám)
 - *total_amount*: A rendelés teljes végösszege
 - Adattípus: DECIMAL (tizedespontos szám)
 - *status*: A rendelés állapota
 - Adattípus: ENUM (pending, paid, shipped, delivered, cancelled)
 - *payment_method*: Fizetési mód
 - Adattípus: VARCHAR (szöveg)
 - *shipping_address*: Szállítási cím
 - Adattípus: TEXT (szöveg)
 - *order_date*: Rendelés dátuma
 - Adattípus: DATETIME (dátum)

- *is_deleted*: Soft-delete jelző, a rendelés “törölve van-e”
 - Adattípus: TINYINT (igaz/hamis)
5. **order_items tábla**: A rendelések részletezése, az egyes tételek rögzítése
- *id*: Elsődleges kulcs
 - Adattípus: INT (szám)
 - *order_id*: Idegen kulcs a fő rendeléshez
 - Adattípus: INT (szám)
 - *product_id*: Idegen kulcs a megrendelt termékhez
 - Adattípus: INT (szám)
 - *quantity*: A rendelt mennyiség
 - Adattípus: INT (szám)
 - *status*: Rendelt termék állapota
 - Adattípus: INT (szám)
6. **cart_items tábla**: A bejelentkezett felhasználók aktuális kosár tartalmának adatai
- *id*: Elsődleges kulcs
 - Adattípus: INT (szám)
 - *customer_id*: Idegen kulcs a felhasználóhoz
 - Adattípus: INT (szám)
 - *product_id*: Idegen kulcs a termékhez
 - Adattípus: INT (szám)
 - *quantity*: A kosárba tett mennyiség
 - Adattípus: INT (szám)
7. **reviews tábla**: A termékekhez tartozó vásárlói értékelések
- *id*: Elsődleges kulcs
 - Adattípus: INT (szám)
 - *product_id*: Idegen kulcs a termékhez
 - Adattípus: INT (szám)
 - *user_id*: Idegen kulcs a felhasználóhoz
 - Adattípus: INT (szám)

- *rating*: Termék értékelése
 - Adattípus: INT (szám)
- *comment*: Szöveges vélemény
 - Adattípus: TEXT (szöveg)
- *created_at*: Értékelés beküldésének ideje
 - Adattípus: TIMESTAMP (idő)

8. **support_tickets** tábla: Az ügyfélszolgálati rendszer adatai.

- *id*: Elsődleges kulcs
 - Adattípus: INT (szám)
- *question*: A felhasználó kérdése
 - Adattípus: TEXT (szöveg)
- *answer*: Admin válasza
 - Adattípus: TEXT (szöveg)
- *status*: A jegy állapota
 - Adattípus: VARCHAR (szöveg)
- *created_at*: Jegy beküldésének a dátuma
 - Adattípus: DATETIME (dátum)

Egységtesztek

A manuális tesztelés mellett a *Vendora* backend rendszere automatizált egységtesztekkel is rendelkezik. A tesztelés célja az egyes metódusok és logikai egységek külön vizsgálata, biztosítva, hogy a kód egy későbbi módosítás során is megőrizze helyes működését.

Alkalmazott technológiák:

- **xUnit**: A .NET legnépszerűbb és legmodernebb tesztkeretrendszere, amelyeket a tesztesetek létrehozására és futtatására használtunk.
- **Moq**: A tesztelés során a valós adatbázis-kapcsolat helyett egy szimulált, csak a memóriában futó környezetet használtunk. Ez lehetővé tette, hogy virtuális adatokon teszteljük a vezérlők logikáját anélkül, hogy ténylegesen íránk az adatbázisba.
- **AAA minta**: Minden tesztesetünket a szakmában elfogadott hármas tagolás szerint építettük fel:
 - **Arrange**: A tesztkörnyezet és a bemeneti adatok előkészítése.
 - **Act**: A vizsgált metódus meghívása.

- **Assert:** Az kapott eredmény összehasonlítása az elvárt kimenettel.

Néhány implementált tesztesetünk:

Tesztelt funkció	Tesztelési forgatókönyv	Elvárt eredmény
Rendelés leadása (Bankkártya)	Érvényes kosárral és bankkártyás fizetési móddal rendelkező felhasználó rendelést ad le	HTTP 200 (Ok), a kosár kiürül, a rendelés státusza automatikusan <i>Paid</i> lesz.
Kosár mennyiség módosítása	A felhasználó megváltoztatja egy már a kosárban lévő termék darabszámát	HTTP 200 (Ok), az adatbázisban az adott tétel mennyisége szinkronizálódik az új értékkel
Kategória törlése	Adminisztrátor próbál törölni egy kategóriát, amelyben még vannak termékek	HTTP 400 (BadRequest), a törlés elutasítva, hibaüzenet küldése
Termékmódosítás (Védett)	Egy eladó megpróbálja egy másik eladóhoz tartozó termék adatait frissíteni	HTTP 403 (Forbidden), a módosítás elutasítva, az eredeti adatok sértetlenek maradnak
Ügyfélszolgálati válasz	Adminisztrátor egy nem létező Support Ticket azonosítóra próbál választ küldeni	HTTP 404 (NotFound), a rendszer jelzi, hogy az adott erőforrás nem található

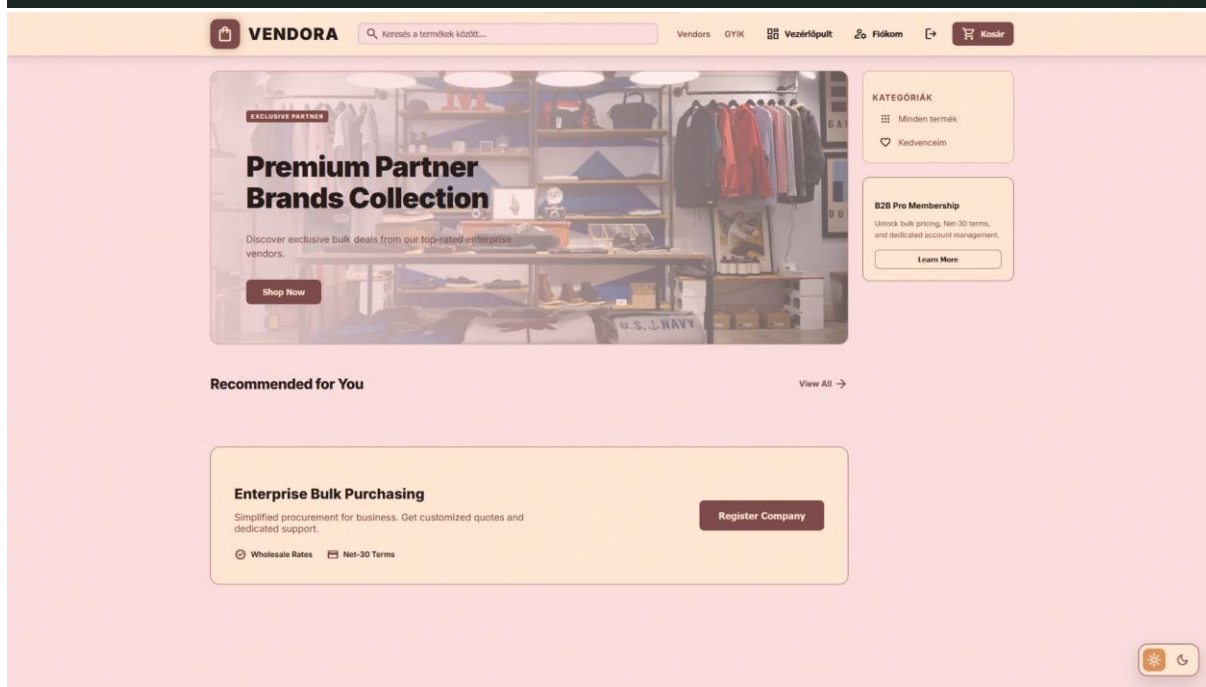
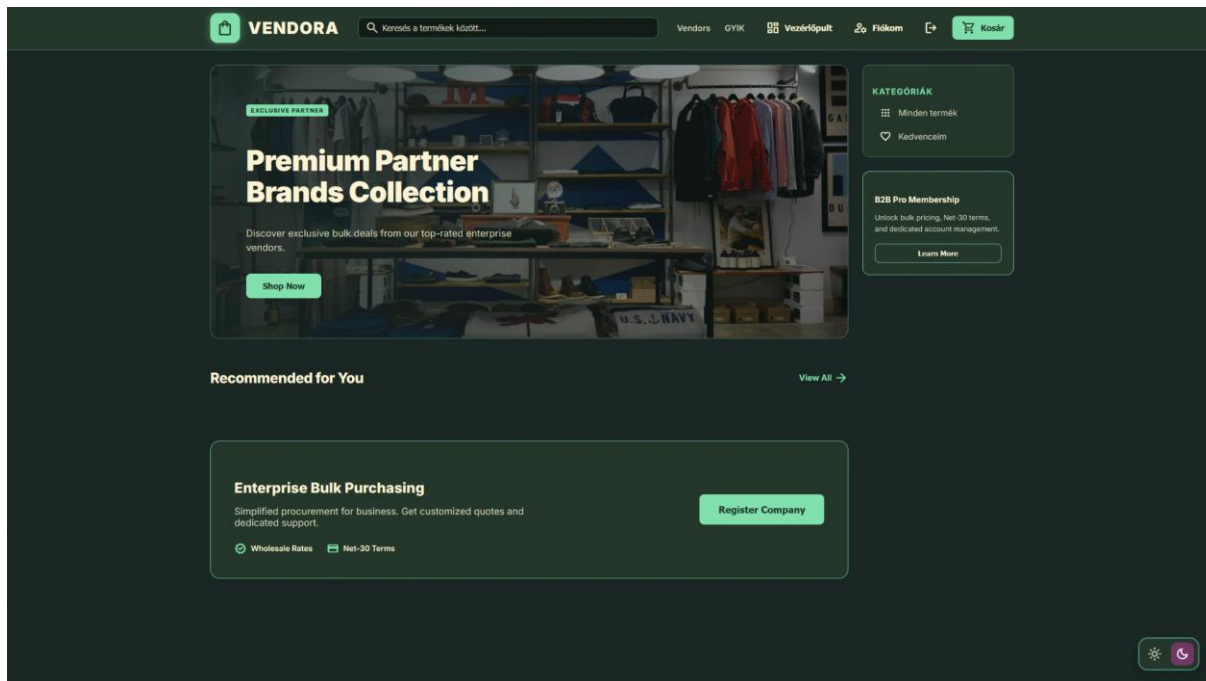
Az egységtesztek stabil alapot és folyamatos visszajelzést biztosítanak a fejlesztés során. Ez garantálja, hogy az üzleti logika (például a fizetési folyamatok) minden körülmények között hibamentesen működjön.

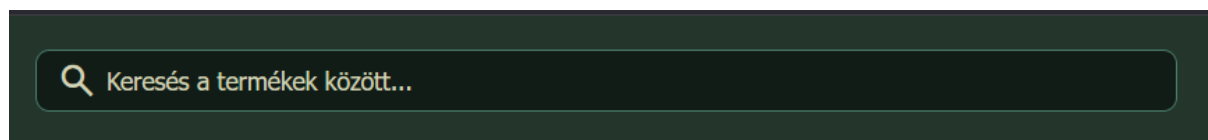
Monitor Képek

Főoldal

Weboldal főoldala sötét és világos megjelenítés

A felhasználó bármelyik oldalon a fejlécben tud navigálni a kosár, profil kezelőgombok (vezérlőpult/fiókom, kijelentkezés), GYIK/FAQ, vendors(eladók) gombokkal, a fejléc bal oldalán a VENDORA címre és egyben gombra kattintva visszakerül a főoldalra.

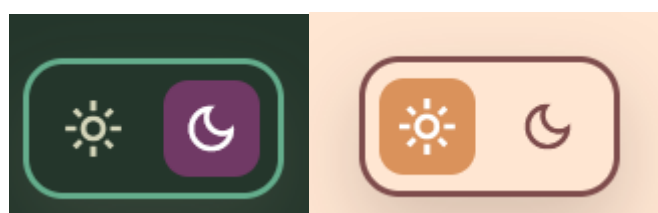




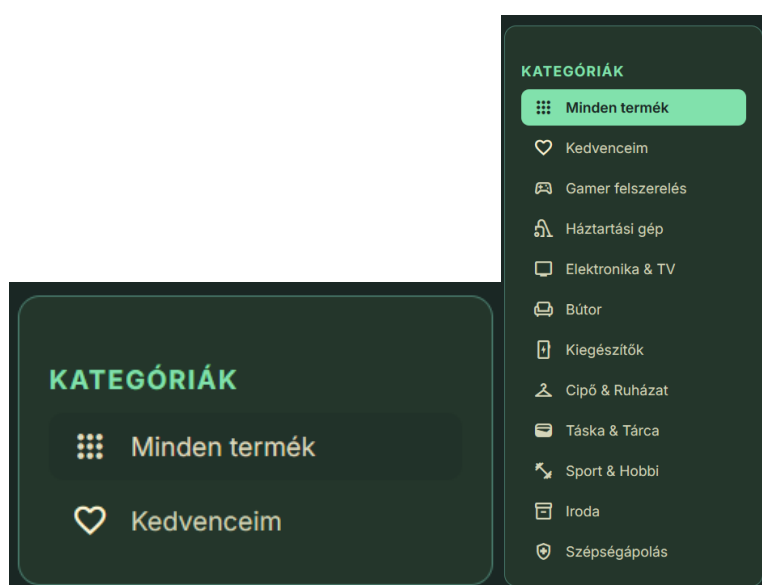
A keresőmezőben termékekre tud keresni a felhasználó, ENTER lenyomása után egyből a /browse című termékek oldalra irányítja a felhasználót csak a szűrés eredményének megfelelő találatokat megjelenítve.



A webshop minden oldalának alján megtalálható a lábléc, ami szintén megkönnyíti az oldalak közti navigációt, a címek beszédes nevekkkel rendelkeznek hogy egyértelmű és gyors legyen használatuk.



A képernyő jobb alsó sarkában található a megjelenítés mód gomb, amivel a felhasználó választhatja a weboldal megjelenését, megpróbáltuk a legszemkímélőbb színeket választani, hogy ne az adott fény viszonyoknak megfelelően a szemnek a lehető legkíméletesebb és legkellemesebb legyen a látvány.



A “KATEGÓRIÁK” menü részen tud a felhasználó a terméket között válogatni kategóriákra bontva.

A szűrési menü a felvett/törölt kategóriákat követve reszponzívan változik.

A kategóriák gomb alatt található a kedvencek menü gomb, ami “megszívezett” (kedvencként jelölt) termékeket jeleníti meg, ez a menü kívánság lista funkciót látja el

Fiókom menü

Ez a menü felel a regisztrált fiók adatok megjelenítésére, itt tekintheti meg a felhasználó fiókja adatait, törölheti a fiókot, nyomon követheti rendeléseit és szerkesztheti adatait.

Személyes Fiókom

[Adataim](#)[Rendeléseim](#)



Teszt Fiók
teszt@fiok.com
Vásárló

Profil szerkesztése

KERESZTNÉV
Teszt

VEZETÉKNÉV
Fiók

EMAIL
teszt@fiok.com

Veszélyes Zóna

A fiók törlése végleges. Minden adatod és megrendelésed elvész.

Fiók végleges törlése

Vezérlőpult/Termékeim (Admin/Vendor jogosultság)

A vezérlőpult segíti az rendszeradminisztrátorok és eladók feladatát, egyszerűsíti a termékek kezelését, rendelések nyomonkövetése, fiókok számontartása, és GYIK/FAQ kérdések megválaszolása.

Az **Áttekintés** menüben láthatunk eladói statisztikákat, figyelhetjük boltunk állapotát.

Az **Összes Rendelés** és **Beérkezett Rendelések** menük, ugyanaz az érme két oldala, az elsőt kizárólag az admin látja, hogy figyelemmel tudja tartani az oldal rendelési forgalmat, a másodikat mindkét fél látja de különböző célból, a vendor a boltjának forgalmát tudja kezelni az oldalon, az admin pedig rendszertesztelés céljából éri el a menüt.

The image displays two screenshots of a web application interface, likely for a vendor or admin dashboard.

Top Screenshot (Vendor Admin):

- Header:** "VENDORA ADMIN" and "Vezérlőpult".
- Left Sidebar:** Contains navigation links: "Áttekintés" (highlighted), "Összes Rendelés", "Beérkezett Rendelések", "Termékek", "Fiók", "Fiókom", and "Support Q&A".
- Main Content Area:** Displays a greeting "Szia, vegre!" and a sub-header "Itt van egy gyors áttekintés az üzleted jelenlegi állásáról." Below this are four summary cards: "TELJES BEVÉTEL" (0 Ft), "ELADOTT TERMÉKEK" (0 db), "VÁRAKOZÓ TÉTELEK" (0 db), and "AKTÍV TERMÉKEID" (120 db).
- Footer:** Shows the user's email "vegre@mukodik.hu" and a "Kijelentkezés" button.

Bottom Screenshot (Vendor Dash):

- Header:** "VENDORA DASH" and "Vezérlőpult".
- Left Sidebar:** Contains navigation links: "Áttekintés", "Beérkezett Rendelések" (highlighted), "Termékek", "Fiók", and "Support Q&A".
- Main Content Area:** Displays the title "Beérkezett Rendelések" and a subtitle "A rendelések állapota és részletei." Below this is a table with columns: "RENDELÉS ID", "DÁTUM", "VÁSÁRLÓ", "ÉRTÉK", and "MŰVELETEK". The table is currently empty, showing "Nincsenek rendelések."
- Footer:** Shows the user's email "tesztelekveg@gmail.com" and a "Kijelentkezés" button.

A **Termékek** oldal hozzáférést biztosít az adminoknak az összes termékhez, itt tudják szerkeszteni és állítani a termék státuszát (*is_approved/is_deleted*).

ID	NÉV	FELTÖLTŐ (VENDOR)	ÁLLAPOT	MŰVELETEK
#162	Férfi Parfüm	Vendora Admin ID: 1	Publikus	▼ ✎ 🕒 🗑️

Termék Szerkesztése

Termék neve

Leírás

Termék Képe

 Kép feltöltése

Előnézet:


Ár (Ft)

Készlet (db)

Kategória

Változtatások mentése

A **Fiókok** menüben tud az adminisztrátor jogkörrel rendelkező felhasználó minden fiókot átlátni és kezelni, itt belelát a fiók adataiba és ezeket tudja szerkeszteni kezelni, jogkört átírni.

Felhasználói Fiókok					
Platform összes regisztrált felhasználójának kezelése.					
Keresés név/email...			Minden szerepkör		
ID / DÁTUM	NÉV / CÉGNÉV	EMAIL	SZEREPKÖR	STÁTUSZ	MŰVELETEK
#1 08/04/2026	Vendora Admin	vegre@mukodik.hu	Admin	Aktív	 
#20 26/04/2026	doma Fiók	doma2@vendora.hu	Customer	Aktív	  
#21 26/04/2026	Teszt Fiók	teszt@fiok.com	Customer	Aktív	  

Felhasználó Szerkesztése

Keresztnév

Teszt

Vezetéknév

Fiok

Cégnév (Opcionális)

Szerepkör

Vásárló (Customer)

Fiók mentése

A **Fiókom** menü hasonló mint amit a NavBarból lehet elérni, viszont annyi különbséggel, hogy visszajelzi a felhasználónak a jogkörét és tokenjének érvényességét

vegre@mukodik.hu

Szerepkör: Admin

AZONOSÍTÁSI TOKEN

Érvényes (Aktív)

JOGOSULTSÁGOK

Teljes platform kezelése (Admin)

Elérkeztünk az utolsó menüponthoz, **Support Q&A**, beszédes neve van, itt jelennek meg a GYIK/FAQ kérdések, ezt szántuk a közérdekű információk az oldallal kapcsolatban elérhetővé tételének, felhasználók oldallal vendőrral és egyebekkel kapcsolatban tehetnek fel kérdést vagy

az oldal vezetősége tud kitenni köz információt pl.(2026.05.03 15:00 és 16:00 között karbantartás miatt nem lesz elérhető a weboldal).

Regisztráció

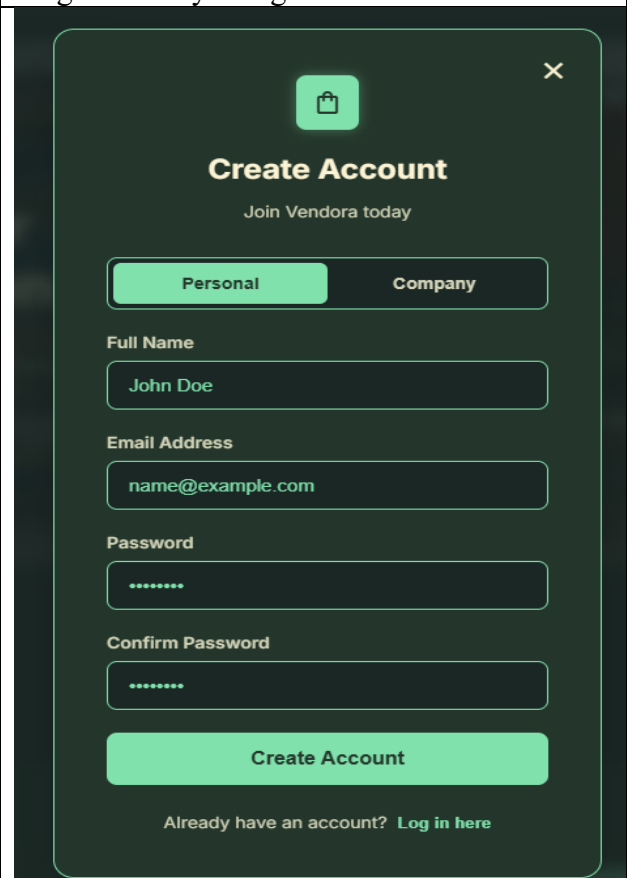
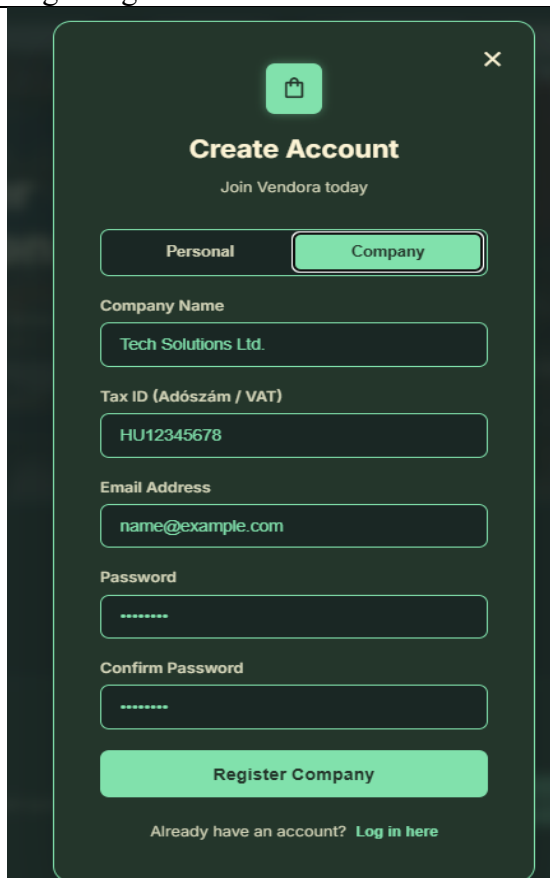
A látogató a weboldal felkeresésekor, vásárlását megelőzően dönthet a regisztrációjáról.

A folyamat elején kiválaszthatja, hogy magánszemélyként, vagy céggént kíván regisztrálni.

- **Magánszemélyként** történő regisztráció esetén meg kell adnia a teljes nevét, egy érvényes e-mail cím és egy erős jelszó megadása szükséges. A biztonság érdekében a jelszót két külön mezőben kell megadni, így elkerülhetők az elgépelésből adódó hibák.
- **Céggént** történő regisztráció esetén, meg kell adni a cég nevét, adószámát, egy email címet és egy jelszót: Ezt követően az admin fogadja el a regisztrációt. Amennyiben a jóváhagyás nem történik meg, abban az esetben a belépés nem engedélyezett.

A **Regisztráció** gombra kattintva - amennyiben a megadott adatok megfelelőek, illetve az admin elfogadta a kérést – létrejön az új fiók, melyre belépve a vásárló nyomon követheti rendelését, később visszakeresheti a korábbi megrendeléseit.

Ha a felhasználónak már van meglévő fiókja, az ablak alján található Bejelentkezés hivatkozással közvetlenül a bejelentkezés ablakra navigálhat. Az ablak jobb felső sarokban lévő X gombbal bármikor bezárható, visszatérve a főoldalra.

Magánszemélyek regisztrációs felülete:	Cégek regisztrációs felülete:
	

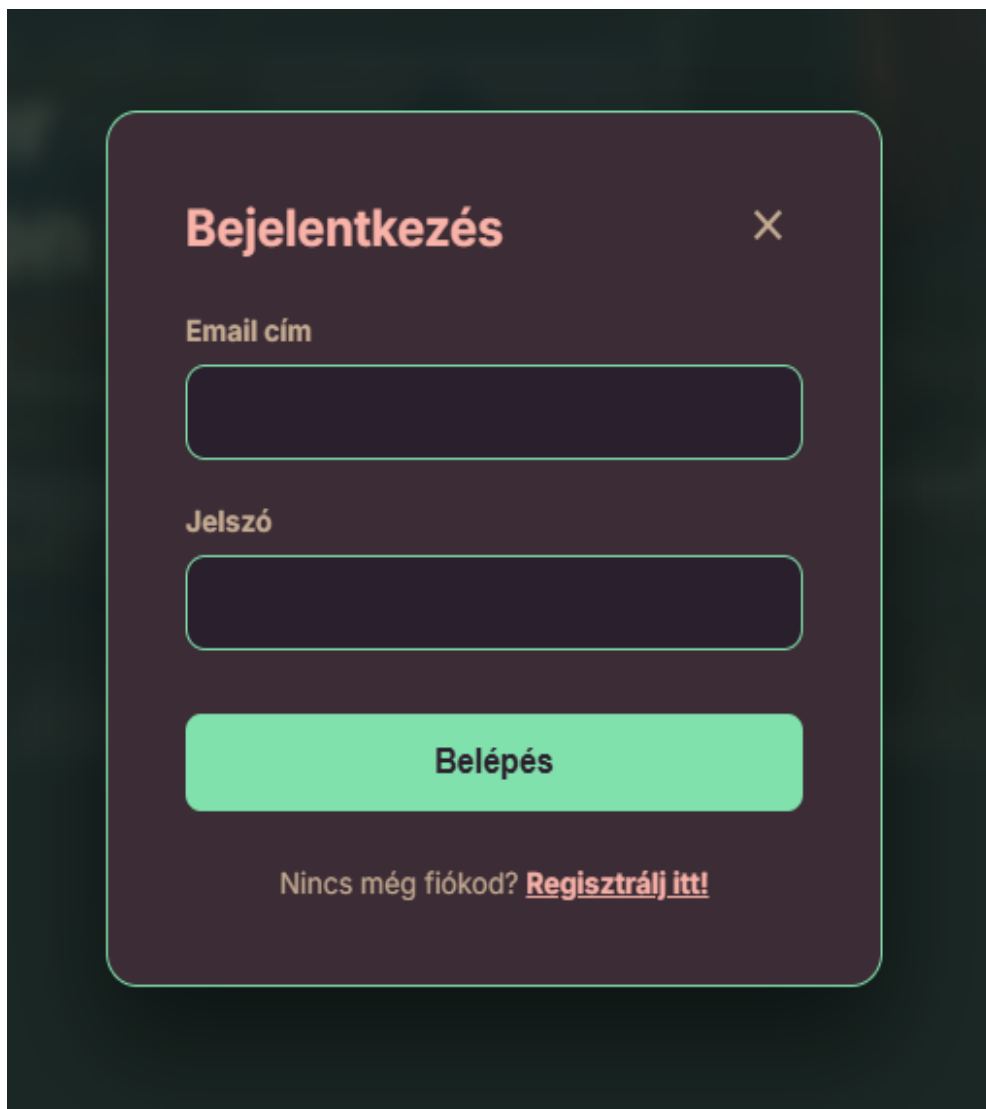
Bejelentkezés

Ha a felhasználó már regisztrált tagja a Vendorának, ezen a felületen keresztül léphet be a fiókjába.

A belépéshez az e-mail cím és a jelszó megadása szükséges.

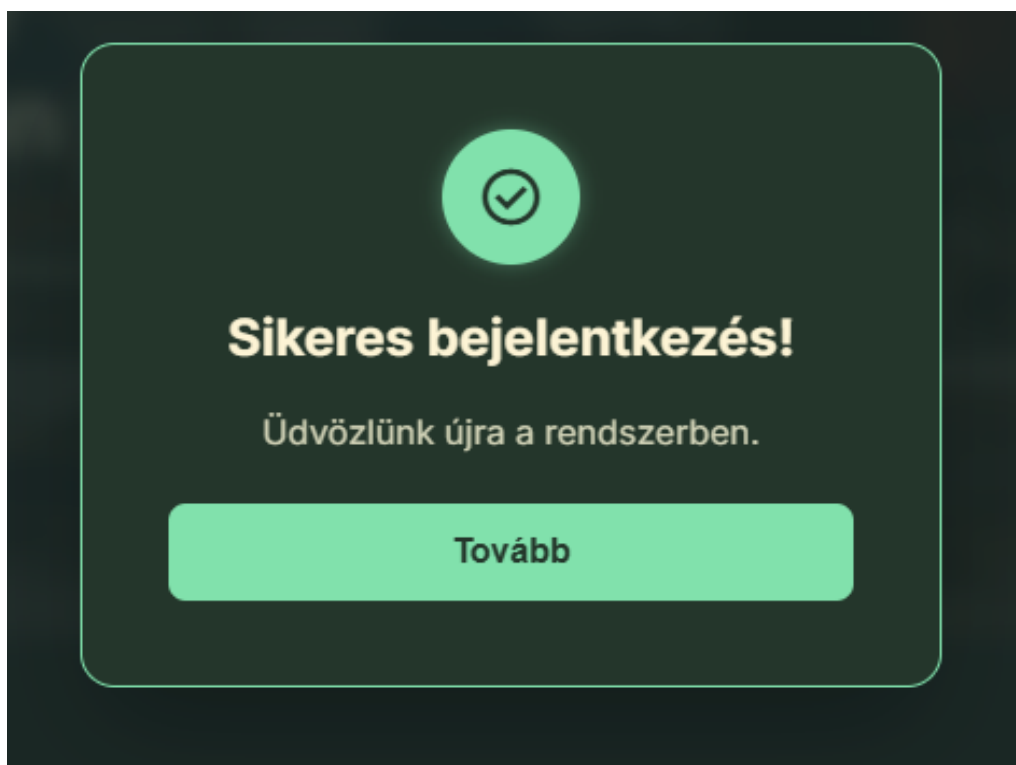
A rendszer a bejelentkezés során ellenőrzi a megadott adatok helyességét. Ha valamelyik adat nem egyezik, a felület jól látható hibaüzenettel tájékoztatja a felhasználót a sikertelen bejelentkezésről.

Az ablak alján található link segítségével a még nem regisztrált látogatók egyetlen kattintással átválthatnak a regisztrációs ablakra. A navigáció megkönnyítése érdekében az **X** gombbal a felhasználó bármikor visszatérhet az áruház kezdőoldalára.



The image shows a login form titled "Bejelentkezés" (Login) with a close button (X) in the top right corner. The form has a dark purple background. It contains two input fields: "Email cím" (Email address) and "Jelszó" (Password). Below the input fields is a large green button labeled "Belépés" (Login). At the bottom of the form, there is a link that says "Nincs még fiókod? [Regisztrálj itt!](#)" (Don't have an account? [Register here!](#)).

Miután a rendszer ellenőrizte a megadott adatokat és a bejelentkezés sikeresen megtörtént, a felhasználót egy új ablak fogadja. Ez visszajelzést ad a felhasználónak, hogy sikerült a bejelentkezés. A felhasználó a **Tovább** gombbal zárhatja be az ablakot, és megkezdődhet a vásárlás.



Összegzés

A Vendora webáruház készítése során igyekeztünk modern választ adni a mai e-kereskedelmi piac kihívásaira, mind esztétikai megjelenésében, mind technológiai háttérében naprakész megoldásokat kínál az oldal felkeresőinek.

Olyan platformot alkottunk meg, mely a vásárlóknak a túlzsúfolt felületek helyett letisztult, átlátható, könnyen értelmezhető felületeket, modern design megjelenítést, valamint a kimagasló sebesség révén a magas vásárlói élményt biztosít. Mindezek mellett az eladók és adminisztrátorok számára egy funkcionális, jól átlátható és megbízható platformot biztosít a kereskedelmi folyamatok hatékony kezeléséhez.

Az oldal készítése közben több problémába is ütköztünk (kommunikációs hiba, github hibás használata, stb.), de ezeken mind sikerült túllendülnünk. Sok információt és tudást szereztünk a weboldal, backend és adatbázis készítés témakörében, türelmesebbek és elfogadóbbak lettünk egymással, megtanultunk csapatban dolgozni.

Források

YouTube	-	https://www.youtube.com/
Stack Overflow	-	https://stackoverflow.com/
W3Schools	-	https://www.w3schools.com
Unsplash	-	https://unsplash.com
Gemini	-	https://gemini.google.com/
Google Fonts	-	https://fonts.google.com
Picsum	-	https://picsum.photos
Pinterest	-	https://hu.pinterest.com

Továbbfejlesztési lehetőségek

Többnyelvűség: A felület és a termékadatok lefordítása több nyelvre (pl. angol, német), lehetővé téve a webshop nemzetközi terjeszkedését.

Több Pénznem: Az árak automatikus átváltása a felhasználó régiója alapján (pl. EUR, USD).

Termékösszehasonlító: Lehetővé teszi több hasonló termék műszaki adatainak és árának összehasonlítását.

Eladói Dashboard bővítése: Részletesebb kimutatások az eladók számára: melyik terméküket nézték meg a legtöbben, várhatóan mennyi lesz a jövő havi bevételük a trendek alapján stb.

Rendeléskövetés: A vevő valós időben tudja követni a rendelését

Rólunk: Egy dedikált aloldal létrehozása, amely bemutatja a Vendra történetét és a csapat értékeit.

B2B partnerek: Egy speciális üzleti felület, amely lehetővé teszi a nagykereskedelmi mennyiségek kezelését, egyedi partneri árazást.

Kapcsolat: Interaktív ügyfélszolgálati modul, amely tartalmaz egy kapcsolatfelvételi űrlapot, térképes megjelenítést a telephelyhez, valamint közvetlen elérést a **Support** oldalhoz.

Adatvédelmi irányelvek: Tájékoztatás az adatok gyűjtéséről, tárolásáról és a felhasználók jogairól.

Live Support: Egy valós idejű csevegőablak, amely lehetővé teszi a vásárlók számára, hogy azonnali segítséget kérjenek az eladóktól vagy az adminisztrátoroktól.

Szállító cégek megadása: Az eladók szabadon választhatnak különböző futárszolgálatok közül.

Csomagpontok: Térképes választó felület amely lehetővé teszi a vásárlók számára, hogy ne csak házhoz szállítsák a termékeket, hanem a csomagpontokra is tudják rendelni a csomagjaikat.

Fizetési mód: Stripe vagy Barion integráció.

Visszaigazoló email küldése: Rendelés leadásakor a vevő emailben kap értesítést a rendeléséről.